

Running funconstrain tests in package optimx

John C. Nash (profjcnash at gmail.com)

2024-04-08

Abstract

The **funconstrain** package (<https://github.com/jlmelville/funconstrain>) provides R users with a convenient tool to access the test functions of Moré et al. [1981]. This vignette article describes a program to apply these test functions to solvers in the **optimx** package (Nash and Varadhan [2011]).

Background

Numerical optimization of functions of several, namely **n**, parameters is an important computational task. R (R Development Core Team [2008]) is a major platform for scientific and statistical calculations and has provided tools for numerical optimization and nonlinear least squares since its inception. These have been extended via a number of packages. In particular, the author has been heavily involved in this effort, and in collaboration with others has provided the package **optimx** which wraps a number of solvers to allow their invocation by a common calling syntax. Note that *optimization* in R generally means *function minimization*, possibly with bounds (or box) constraints on the function parameters.

It is extremely helpful to users to have examples and tests of function minimization. In many situations it is extremely easy to insert an error into code, so easy-to-apply tests allow for the discovery of such errors. There are a number of collections of test functions with many overlaps and minor differences. A well-established and well-documented set of such functions are those of Moré et al. [1981]. These have been translated into R by James Melville in the R package **funconstrain** (<https://github.com/jlmelville/funconstrain>). While initially these provided the function and its gradient given a set of suitable input parameters, the present author added code to compute the Hessian for each test function. This allows Newton-like solvers to be applied. **funconstrain** also provides suggested initial parameter vectors for each of the 35 test functions. However, where there are multiple input possibilities, just one is provided, for example when the test function has a variable number of parameters.

What is then missing is the link between **funconstrain** and the tools in **optimx**, which this article aims to provide.

Function fufn()

Most of the test functions in Moré et al. [1981] are sums of squares of nonlinear functions. While **n** is the number of parameters, we may have a different number of functions squared in the summation. Call this **m**. This may be altered to give different variations of a given function, so **m** must be provided.

Many of the solvers in **optimx** are capable of handling bounds constraints on the **n** parameters. That is parameter **i** must satisfy

$$\text{lower}[i] \leq \text{prm}[i] \leq \text{upper}[i]$$

where **prm** is the parameter vector and **lower** and **upper** are vectors of numbers providing lower and upper bounds. Methods in **optimx** that can handle masks are listed in the character vector **bdmeth** returned by the function **optimx::ctrldefault(n)**. Note that a number of parameters **n** must nominally be provided to **ctrldefault()** but generally **n** can be specified as 2 to get the default settings for **optimx**. At time of writing

```
bdmeth <- c("L-BFGS-B", "nlinb", "lbfgsb3c", "Rcgmin", "Rtnmin", "nvm",
           "Rvmmin", "bobyqa", "nmkb", "hjkb", "hjn", "snewtonm", "ncg",
           "slsqp", "tnewt", "nlm", "snewtm", "spg")`
```

Note that to use the `lbfgsb3c`, and `lbfgs` methods, you must install the `lbfgsb3c` and `lbfgs` packages.

If the upper and lower bound for a parameter are equal, we can say the parameter is **fixed** or **masked**. This may seem to be a silly option, since it essentially reduces the dimensionality of the problem. However, there are many situations where we have evidence that a parameter takes a particular (fixed) value, but know that we may wish to allow optimization over that parameter in later investigations. Masks allow us to avoid having to rewrite the function, gradient and Hessian code. However, only a few optimization solvers handle masks. The function `optimx::ctrldefault()` returns a value `maskmeth` with a list of solvers that do handle the situation where lower and upper bounds coincide. At the time of writing this is specified as

```
maskmeth <- c("Rcgmin", "nvm", "hjn", "ncg", "snewtonm", "nlinb", "L-BFGS-B")
```

With the above in mind, the function `fufn()` was written to access the test functions of `funconstrain`.

Calling `fufn()`

The function `fufn()` takes one parameter, the numeric value of the test that you want parameters for. See Appendix A below for a list of the test function names and numbers. For example, running `fufn(1)` will return parameters for `rosen()`.

While we can write our own driver for `fufn()`, I wanted to make the task extremely easy. Thus the function `fufnrun` is provided. This is set up to use a simple text file, `RFO.txt`, to specify which test functions are to be applied to which solvers. Moreover, a “sink” file name can be specified to save the text output of the run.

Test specification file `RFO.txt`

Let us consider an example.

```
testsink240408A.txt
1, 9, 9, 1, 6:8, 35
c("L-BFGS-B", "lbfgs", "lbfgsb3c", "lbfgs")
FALSE
```

The lines of the above file provide the following information:

- the first line is the name of the text file to use to save the output via `sink()`.
- line 2 says that test functions 1, 6, 7, 8, and 35 are to be used. Note that we can use the colon “:” when giving a contiguous range of function numbers. These numbers – by referring back to the vector `funnam` at the top of function `fufn()` – specify functions “rosen”, “jenn-samp”, “helical”, “bard” and “chebyquad”. Using the function numbers. Appendix A lists the numbers and corresponding names. The specification `1:35` uses all test functions. The program removes duplicate problem numbers and sorts the list in ascending order.
- line 3 gives an R character vector of the solver methods to be applied. At the time of writing, there is no check for duplicate entries in the vector.
- line 4 is `TRUE` if the experimental bounds constraints are to be applied.

A driver program for `fufn()`

The `fufnrun` function is a driver for `fufn()`. It takes one parameter, the path to a file (by default `RFO.txt`) containing the specification above. It reads the file and then calls `fufn()` with the specified parameters. If you save the output above to a file, e.g. `path/to/RFO.txt` and ensure `optimx`, `lbfgs` and `lbfgsb3c` are installed, run:

```
fufnrun("/path/to/RFO.txt")
```

and the results of the evaluation will be logged to the console.

Using the Hessian

`funconstrain` can generate the Hessian function for the test problems. The following specification script will run all problems using three solvers capable of taking advantage of the Hessian.

```
testsink230410A.txt
1:35
c("nlm", "nlminb", "snewtm")
FALSE
```

The classic WOOD test function returns results

```
Problem: wood
      p1 s1 p2 s2 p3 s3 p4 s4      value fevals gevals hevals conv kkt1 kkt2 xtime
nlminb  1   1   1   1   1 6.637402e-29    55    44    44    0 TRUE TRUE 0.001
snewtm  1   1   1   1   1 3.930599e-27    71    49    48    0 TRUE TRUE 0.004
nlm     1   1   1   1   1 1.004941e-16   354   354   354    0 TRUE TRUE 0.005
END : wood
```

Here we see different performance of three methods. Method `snewtm` is a stabilized Newton method which is part of package `optimx`. While intended mainly as a didactic exercise, this solver has done well on this problem.

Bounded parameters

We can also try the same problems with the experimental bounds constraints via the specification script:

```
testsink230410B.txt
1:35
c("nlm", "nlminb", "snewtm")
TRUE
```

For the WOOD function, the results are now

```
Problem: wood
Non-bounds methods requested: [1] "nlm"
      p1 s1 p2 s2 p3 s3 p4 s4      value fevals gevals hevals conv kkt1 kkt2 xtime
nlminb -0.9 U -0.9 U -0.9 U -0.9 U 707.199     7     6     6    0 FALSE TRUE 0.000
snewtm -0.9 U -0.9 U -0.9 U -0.9 U 707.199     6     5     4    0 FALSE TRUE 0.001
END : wood
```

Note that method `nml` is not set up to handle bounds and is automatically dropped by function `opm()`. We also see that the solution found (in both cases) is at the upper bound on all parameters, which is indicated by the status (i.e. “s”) columns of the output table.

Appendix A: function numbers and names

```
1      rosen
2      freud_roth
3      powell_bs
4      brown_bs
5      beale
6      jenn_samp
7      helical
8      bard
9      gauss
10     meyer
11     gulf
```

```
12    box_3d
13    powell_s
14    wood
15    kow_osb
16    brown_den
17    osborne_1
18    biggs_exp6
19    osborne_2
20    watson
21    ex_rosen
22    ex_powell
23    penalty_1
24    penalty_2
25    var_dim
26    trigon
27    brown_al
28    disc_bv
29    disc_ie
30    broyden_tri
31    broyden_band
32    linfun_fr
33    linfun_r1
34    linfun_r1z
35    chebyquad
```

References

References

- Jorge J. Moré, Burton S. Garbow, and Kenneth E. Hillstom. Testing unconstrained optimization software. 7 (1):17–41, March 1981. ISSN 0098-3500 (print), 1557-7295 (electronic).
- John C Nash and Ravi Varadhan. *optimx: A Replacement and Extension of the optim() function*. Nash Information Services Inc. and Johns Hopkins University, 2011. R package version 2011-3.5.
- R Development Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2008. URL <http://www.R-project.org>. ISBN 3-900051-07-0.